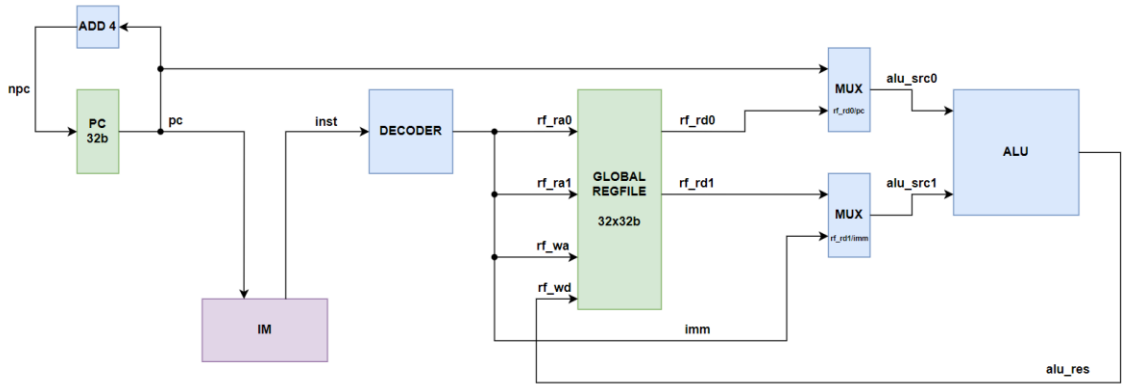
Lab 3 report

实验目的与内容

1. 实验目的：通过 Verilog 硬件描述语言实现一个最为基础的单周期 CPU，本次实验将使用 Verilog 实现一个没有分支、数据访存功能的 CPU。
2. 实验内容：根据选择的指令集设计译码器并且以此搭建一个简单的 CPU。

逻辑设计

1. 各模块框图&数据通路：



2. 状态机：无

任务 1：译码器设计（3 分）

请根据自己选择的指令集，设计译码器 Decoder 模块，以正确生成控制信号。你可以结合仿真证明自己的设计。

代码：

```
module DECODER (

    input                   [31 : 0]            inst,



    output      reg             [ 4 : 0]            alu_op,

    output      reg             [31 : 0]            imm,
```

```
    output      reg             [ 4 : 0]            rf_ra0,

    output      reg            [ 4 : 0]             rf_ra1,

    output      reg           [ 4 : 0]              rf_wa,

    output      reg             [ 0 : 0]            rf_we,
```

```
    output      reg             [ 0 : 0]            alu_src0_sel,

    output      reg             [ 0 : 0]            alu_src1_sel,
```

```
    output      reg             [ 3 : 0]            dmem_access,//内存访问类型

    output      reg             [ 1 : 0]            rd_wd_sel,//0:pc+4,1:alu,2:mem,寄存器堆写入数据选择

    output      reg             [ 3 : 0]            br_type,//

    output      reg             [ 0 : 0]            dmem_we//是否写内存

);
```

```
always @(*)begin

    // Reset outputs

    alu_op = 5'b00000;

    imm = 32'b0;

    rf_ra0 = inst[9:5];

    rf_ra1 = inst[14:10];

    rf_wa = inst[4:0];

    rf_we = 1'b0;

    alu_src0_sel = 1'b0;

    alu_src1_sel = 1'b0;
```

```

    casez(inst[31:15])

        17'b00000000000100000:begin //add.w

            alu_op = 5'b00000;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;//选寄存器

            alu_src1_sel = 1'b0;//选寄存器

            imm = 32'b0;

            dmem_access = 4'h8;//NONE

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu
```

```
            dmem_we = 1'b0;//不写内存
        end
        17'b0000000000100010:begin//sub.w
            alu_op = 5'b00010;
            rf_we = 1'b1;
            alu_src0_sel = 1'b1;
            alu_src1_sel = 1'b0;
            imm = 32'd0;
            dmem_access = 4'h8;//NONE
            br_type = 4'h8;//NONE
            rd_wd_sel = 2'b01;//alu
            dmem_we = 1'b0;//不写内存
        end

        17'b00000000000100100:begin//slt.w
            alu_op = 5'b00100;
            rf_we = 1'b1;
            alu_src0_sel = 1'b1;
            alu_src1_sel = 1'b0;
            imm = 32'd0;
            dmem_access = 4'h8;//NONE
            br_type = 4'h8;//NONE
            rd_wd_sel = 2'b01;//alu
            dmem_we = 1'b0;//不写内存
        end

        17'b00000000000100101:begin//sltu.w
            alu_op = 5'b00101;
            rf_we = 1'b1;
            alu_src0_sel = 1'b1;
            alu_src1_sel = 1'b0;
            imm = 32'd0;
            dmem_access = 4'h8;//NONE
```

```
            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存



        end
```

```
        17'b00000000000101001:begin//and.w

            alu_op = 5'b01001;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;

            alu_src1_sel = 1'b0;

            imm = 32'd0;

            dmem_access = 4'h8;//NONE

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b00000000000101010:begin//or.w

            alu_op = 5'b01010;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;

            alu_src1_sel = 1'b0;

            imm = 32'd0;

            dmem_access = 4'h8;//NONE

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b00000000000101011:begin//xor.w

            alu_op = 5'b01011;
```

```
            rf_we = 1'b1;

            alu_src0_sel = 1'b1;

            alu_src1_sel = 1'b0;

            imm = 32'd0;

            dmem_access = 4'h8;//NONE

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b00000000000101110:begin//sll.w

            alu_op = 5'b01110;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;

            alu_src1_sel = 1'b0;

            imm = 32'd0;

            dmem_access = 4'h8;//NONE

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b00000000000101111:begin//srl.w

            alu_op = 5'b01111;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;

            alu_src1_sel = 1'b0;

            imm = 32'd0;

            dmem_access = 4'h8;//NONE

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b00000000000110000:begin//sra.w

            alu_op = 5'b10000;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;

            alu_src1_sel = 1'b0;

            imm = 32'd0;

            dmem_access = 4'h8;//NONE

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b0000001010???????:begin//addi.w,

            alu_op = 5'b00000;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;

            alu_src1_sel = 1'b1;//imm

            //立即数进行符号位扩展

            imm = {{20{inst[21]}},inst[21:10]};

            dmem_access = 4'h8;//NONE

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存

        end

        17'b0000001000???????:begin//slti

            alu_op = 5'b00100;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;

            alu_src1_sel = 1'b1;

            //立即数进行符号位扩展

            imm = {{20{inst[21]}},inst[21:10]};

            dmem_access = 4'h8;//NONE
```

```
            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b0000001001???????:begin//sltiu

            alu_op = 5'b00101;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;

            alu_src1_sel = 1'b1;

            //立即数进行符号位扩展

            imm = {{20{inst[21]}},inst[21:10]};

            dmem_access = 4'h8;//NONE

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b0000001101???????:begin//andi

            alu_op = 5'b01001;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;

            alu_src1_sel = 1'b1;

            //立即数进行 0 扩展

            imm = {{20{1'b0}},inst[21:10]};

            dmem_access = 4'h8;//NONE

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b0000001110???????:begin//ori
```

```
            alu_op = 5'b01010;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;

            alu_src1_sel = 1'b1;

            //立即数进行 0 扩展

            imm = {{20{1'b0}},inst[21:10]};

            dmem_access = 4'h8;//NONE

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b0000001111???????:begin//xori

            alu_op = 5'b01011;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;

            alu_src1_sel = 1'b1;

            //立即数进行 0 扩展

            imm = {{20{1'b0}},inst[21:10]};

            dmem_access = 4'h8;//NONE

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b00000000010000001:begin//slli.w

            alu_op = 5'b01110;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;

            alu_src1_sel = 1'b1;

            //ui5

            imm = {{27{1'b0}},inst[14:10]};

            dmem_access = 4'h8;//NONE
```

```
            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b00000000010001001:begin//srli.w

            alu_op = 5'b01111;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;

            alu_src1_sel = 1'b1;

            //ui5

            imm = {{27{1'b0}},inst[14:10]};

            dmem_access = 4'h8;//NONE

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b00000000010010001:begin//srai.w

            alu_op = 5'b10000;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;

            alu_src1_sel = 1'b1;

            //ui5

            imm = {{27{1'b0}},inst[14:10]};

            dmem_access = 4'h8;//NONE

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b0001010??????????:begin//lu12i.w
```

```
            alu_op = 5'b10010;

            rf_we = 1'b1;

            alu_src0_sel = 1'b0;

            alu_src1_sel = 1'b1;//imm

            imm = {inst[24:5],{12{1'b0}}};

            dmem_access = 4'h8;//NONE

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b0001110??????????:begin//pcaddu12i

            alu_op = 5'b00000;

            rf_we = 1'b1;

            alu_src0_sel = 1'b0;//pc

            alu_src1_sel = 1'b1;//imm

            imm = {inst[24:5],{12{1'b0}}};

            dmem_access = 4'h8;//NONE

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b01;//alu

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b0010100010???????:begin//ld.w

            alu_op = 5'b00000;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;//ra0

            alu_src1_sel = 1'b1;//imm

            imm = {{20{inst[21]}},inst[21:10]};

            dmem_access = 4'h0;//LDW

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b10;//mem

            dmem_we = 1'b0;//不写内存
```

```
        end
```

```
        17'b0010100001???????:begin//ld.h

            alu_op = 5'b00000;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;//ra0

            alu_src1_sel = 1'b1;//imm

            imm = {{20{inst[21]}},inst[21:10]};

            dmem_access = 4'h2;//LDH

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b10;//mem

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b0010100000???????:begin//ld.b

            alu_op = 5'b00000;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;//ra0

            alu_src1_sel = 1'b1;//imm

            imm = {{20{inst[21]}},inst[21:10]};

            dmem_access = 4'h4;//LDB

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b10;//mem

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b0010101001???????:begin//ld.hu

            alu_op = 5'b00000;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;//ra0

            alu_src1_sel = 1'b1;//imm
```

```
            imm = {{20{inst[21]}},inst[21:10]};

            dmem_access = 4'h6;//LDHU

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b10;//mem

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b0010101000???????:begin//ld.bu

            alu_op = 5'b00000;

            rf_we = 1'b1;

            alu_src0_sel = 1'b1;//ra0

            alu_src1_sel = 1'b1;//imm

            imm = {{20{inst[21]}},inst[21:10]};

            dmem_access = 4'h7;//LDBU

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b10;//mem

            dmem_we = 1'b0;//不写内存

        end
```

```
        17'b0010100110???????:begin//st.w

            alu_op = 5'b00000;

            rf_ra1 = inst[4:0];

            rf_we = 1'b0;

            alu_src0_sel = 1'b1;//ra0

            alu_src1_sel = 1'b1;//imm

            imm = {{20{inst[21]}},inst[21:10]};

            dmem_access = 4'h1;//STW

            br_type = 4'h8;//NONE

            rd_wd_sel = 2'b11;//none

            dmem_we = 1'b1;//写内存

        end
```

```
17'b0010100101???????:begin//st.h

    alu_op = 5'b00000;

    rf_ra1 = inst[4:0];

    rf_we = 1'b0;

    alu_src0_sel = 1'b1;//ra0

    alu_src1_sel = 1'b1;//imm

    imm = {{20{inst[21]}},inst[21:10]};

    dmem_access = 4'h3;//STH

    br_type = 4'h8;//NONE

    rd_wd_sel = 2'b11;//none

    dmem_we = 1'b1;//写内存

end
```

```
17'b0010100100???????:begin//st.b

    alu_op = 5'b00000;

    rf_ra1 = inst[4:0];

    rf_we = 1'b0;

    alu_src0_sel = 1'b1;//ra0

    alu_src1_sel = 1'b1;//imm

    imm = {{20{inst[21]}},inst[21:10]};

    dmem_access = 4'h5;//STB

    br_type = 4'h8;//NONE

    rd_wd_sel = 2'b11;//none

    dmem_we = 1'b1;//写内存

end
```

```
17'b010011??????????:begin//jirl

    alu_op = 5'b00000;//rj+imm

    imm = {{14{inst[25]}}, inst[25:10], 2'b0};//左移两位后进行符号位扩展

    alu_src0_sel = 1'b1;//ra0

    alu_src1_sel = 1'b1;//imm

    br_type = 4'h6;//JIRL

    dmem_access = 4'h8;//NONE
```

```
            rd_wd_sel = 2'b00;//pc+4

            dmem_we = 1'b0;//不写内存

            if(rf_wa ==5'b00000)

                rf_we = 1'b0;//不写寄存器,非调用跳转

            else

                rf_we = 1'b1;//写寄存器

        end
```

```
        17'b010100??????????:begin//b

            alu_op = 5'b00000;

            imm = {{4{inst[9]}}, inst[9:0],inst[25:10], 2'b0};//左移两位后进行符号位扩展

            alu_src0_sel = 1'b0;//pc

            alu_src1_sel = 1'b1;//imm

            br_type = 4'h6;//B

            dmem_access = 4'h8;//NONE

            rd_wd_sel = 2'b11;//none

            dmem_we = 1'b0;//不写内存

            rf_we = 1'b0;//不写寄存器

        end
```

```
        17'b010101??????????:begin//bl

            alu_op = 5'b00000;

            rf_wa = 5'b00001;//写回一号寄存器

            imm = {{4{inst[9]}}, inst[9:0],inst[25:10], 2'b0};//左移两位后进行符号位扩展

            alu_src0_sel = 1'b0;//pc

            alu_src1_sel = 1'b1;//imm

            br_type = 4'h6;//BL

            dmem_access = 4'h8;//NONE

            rd_wd_sel = 2'b00;//pc+4

            dmem_we = 1'b0;//不写内存

            rf_we = 1'b1;//写寄存器

        end
```

```
        17'b010110??????????:begin//beq

            alu_op = 5'b00000;

            rf_ra1 = inst[4:0];

            imm = {{14{inst[25]}}, inst[25:10], 2'b0};//左移两位后进行符号位扩展

            alu_src0_sel = 1'b0;//pc

            alu_src1_sel = 1'b1;//imm

            br_type = 4'h0;//BEQ

            dmem_access = 4'h8;//NONE

            rd_wd_sel = 2'b11;//none

            dmem_we = 1'b0;//不写内存

            rf_we = 1'b0;//不写寄存器



        end
```

```
        17'b010111??????????:begin//bne

            alu_op = 5'b00000;

            rf_ra1 = inst[4:0];

            imm = {{14{inst[25]}}, inst[25:10], 2'b0};//左移两位后进行符号位扩展

            alu_src0_sel = 1'b0;//pc

            alu_src1_sel = 1'b1;//imm

            br_type = 4'h5;//BNE

            dmem_access = 4'h8;//NONE

            rd_wd_sel = 2'b11;//none

            dmem_we = 1'b0;//不写内存

            rf_we = 1'b0;//不写寄存器

        end
```

```
        17'b011000??????????:begin//blt

            alu_op = 5'b00000;

            rf_ra1 = inst[4:0];

            imm = {{14{inst[25]}}, inst[25:10], 2'b0};//左移两位后进行符号位扩展
```

```
            alu_src0_sel = 1'b0;//pc

            alu_src1_sel = 1'b1;//imm

            br_type = 4'h3;//BLT

            dmem_access = 4'h8;//NONE

            rd_wd_sel = 2'b11;//none

            dmem_we = 1'b0;//不写内存

            rf_we = 1'b0;//不写寄存器

        end
```

```
        17'b011001??????????:begin//bge

            alu_op = 5'b00000;

            rf_ra1 = inst[4:0];

            imm = {{14{inst[25]}}, inst[25:10], 2'b0};//左移两位后进行符号位扩展

            alu_src0_sel = 1'b0;//pc

            alu_src1_sel = 1'b1;//imm

            br_type = 4'h1;//BGE

            dmem_access = 4'h8;//NONE

            rd_wd_sel = 2'b11;//none

            dmem_we = 1'b0;//不写内存

            rf_we = 1'b0;//不写寄存器

        end
```

```
        17'b011011??????????:begin//bgeu

            alu_op = 5'b00000;

            rf_ra1 = inst[4:0];

            imm = {{14{inst[25]}}, inst[25:10], 2'b0};//左移两位后进行符号位扩展

            alu_src0_sel = 1'b0;//pc

            alu_src1_sel = 1'b1;//imm

            br_type = 4'h2;//BGEU

            dmem_access = 4'h8;//NONE

            rd_wd_sel = 2'b11;//none

            dmem_we = 1'b0;//不写内存

            rf_we = 1'b0;//不写寄存器
```

```
        end
```

```
        17'b011010??????????:begin//bltu

            alu_op = 5'b00000;

            rf_ra1 = inst[4:0];

            imm = {{14{inst[25]}}, inst[25:10], 2'b0};//左移两位后进行符号位扩展

            alu_src0_sel = 1'b0;//pc

            alu_src1_sel = 1'b1;//imm

            br_type = 4'h4;//BLTU

            dmem_access = 4'h8;//NONE

            rd_wd_sel = 2'b11;//none

            dmem_we = 1'b0;//不写内存

            rf_we = 1'b0;//不写寄存器

        end
```

```
    endcase

end
```

```
endmodule
```

本题目要求根据选择的指令集设计译码器 Decoder 模块，此处选择 LA32R 指令集。由于此指令集各条指令的 opcode 长度不相等，故采用了优先编码形式 case 语句来识别不同指令，并且在每条指令的 case 对应段里对 alu 运算模式、alu 操作数选择以及立即数的扩展等。

注：由于这个报告是补交的，所以在写的时候 lab3 的各个模块代码已经修改为了 lab4 的代码。

任务 2：搭建 CPU（4 分）

请正确实现 CPU 的各个功能模块，并根据数据通路将其正确连接。**理论上，你只需要完成 CPU 模块及其子模块的设计，而无需修改其他模块的内容。**最终，你需要在 FPGAOL 上上板运行，并通过我们给出的测试程序，为此你需要实现 Lab1 列出的指令中，从 add.w（add）到 pcaddu12i（auipc）之间的全部指令。

CPU 代码：

```
`include "./include/config.v"

`define PC_INIT             32'H1c000000

`define HALT_INST           32'H80000000

module CPU (
```

```
    input                   [ 0 : 0]            clk,

    input                   [ 0 : 0]            rst,

    input                   [ 0 : 0]            global_en,

/* ----------------------------- Memory (inst) ----------------------------- */

    output                  [31 : 0]            imem_raddr,

    input                   [31 : 0]            imem_rdata,

/* ----------------------------- Memory (data) ----------------------------- */

    input                   [31 : 0]            dmem_rdata,

    output                  [ 0 : 0]            dmem_we,

    output                  [31 : 0]            dmem_addr,//output

    output                  [31 : 0]            dmem_wdata,

/* --------------------------------- Debug --------------------------------- */

    output                  [ 0 : 0]            commit,

    output                  [31 : 0]            commit_pc,

    output                  [31 : 0]            commit_inst,

    output                  [ 0 : 0]            commit_halt,

    output                  [ 0 : 0]            commit_reg_we,

    output                  [ 4 : 0]            commit_reg_wa,

    output                  [31 : 0]            commit_reg_wd,

    output                  [ 0 : 0]            commit_dmem_we,

    output                  [31 : 0]            commit_dmem_wa,

    output                  [31 : 0]            commit_dmem_wd,

    input                   [ 4 : 0]            debug_reg_ra,   // TODO

    output                  [31 : 0]            debug_reg_rd    // TODO

);

// TODO

wire [31:0] cur_npc;

wire [31:0] cur_pc, cur_inst, cur_imm;

//assign cur_npc = cur_pc + 4;变成多选了

wire [4:0] cur_rf_ra0, cur_rf_ra1, cur_rf_wa, cur_debug_reg_ra;

wire cur_rf_we;

wire [31:0] cur_rf_wd, cur_debug_reg_rd, cur_rf_rd0, cur_rf_rd1;

wire [0:0]cur_alu_src0_sel, cur_alu_src1_sel;
```

```
wire [4:0] cur_alu_op;

wire [31:0] cur_alu_src0,cur_alu_src1;

wire [3:0] cur_dmem_access;

wire [3:0] cur_br_type;

wire cur_npc_sel;

wire [31:0] cur_rd_out;

wire [1:0] cur_rd_wd_sel;

wire [31:0] cur_alu_res;

assign cur_inst = imem_rdata;

assign debug_reg_rd = cur_debug_reg_rd;

assign cur_debug_reg_ra = debug_reg_ra;

assign imem_raddr = cur_pc;//output

PC my_pc (

    .clk    (clk        ),

    .rst    (rst        ),

    .en     (global_en  ),    // 当 global_en 为高电平时，PC 才会更新，CPU 才会执行指令。

    .npc    (cur_npc    ),

    .pc     (cur_pc     )   // 当前指令的地址

);

DECODER my_decoder (

    .inst   (cur_inst   ),

    .alu_op (cur_alu_op ),

    .imm    (cur_imm),

    .rf_ra0 (cur_rf_ra0),

    .rf_ra1 (cur_rf_ra1),

    .rf_wa  (cur_rf_wa),

    .rf_we  (cur_rf_we),

    .alu_src0_sel (cur_alu_src0_sel),

    .alu_src1_sel (cur_alu_src1_sel),

    .rd_wd_sel (cur_rd_wd_sel),

    .dmem_access (cur_dmem_access),

    .br_type (cur_br_type),

    .dmem_we(dmem_we)
```

```
);
REG_FILE my_regfile(
    .clk    (clk        ),
    .rf_ra0    (cur_rf_ra0 ),
    .rf_ra1    (cur_rf_ra1 ),
    .dbg_reg_ra (cur_debug_reg_ra),
    .rf_wa     (cur_rf_wa  ),
    .rf_we     (cur_rf_we  ),
    .rf_wd     (cur_rf_wd  ),
    .rf_rd0    (cur_rf_rd0 ),
    .rf_rd1    (cur_rf_rd1 ),
    .dbg_reg_rd (cur_debug_reg_rd)
);
MUX  #(
    .WIDTH  (32)
)mux1(
    .src0   (cur_pc),
    .src1   (cur_rf_rd0),
    .sel    (cur_alu_src0_sel),
    .res    (cur_alu_src0)
);
MUX  #(
    .WIDTH  (32)
)mux2(
    .src0   (cur_rf_rd1),
    .src1   (cur_imm),
    .sel    (cur_alu_src1_sel),
    .res    (cur_alu_src1)
);
ALU my_alu (
    .alu_src0   (cur_alu_src0),
    .alu_src1   (cur_alu_src1),
    .alu_op     (cur_alu_op),
```

```
    .alu_res    (cur_alu_res)  // ALU 的输出结果  = 寄存器的写入数据,省略一个寄存器

);

BRANCH my_branch (

    .br_type    (cur_br_type   ),

    .br_src0    (cur_rf_rd0 ),

    .br_src1    (cur_rf_rd1),

    .npc_sel    (cur_npc_sel   )

);

NPC_MUX my_npc_mux (

    .pc_offset  (cur_alu_res   ),

    .npc_sel    (cur_npc_sel   ),

    .npc        (cur_npc   ),

    .pc_add4    (cur_pc + 4)

);

SLU my_slu (

    .addr       (cur_alu_res),

    .dmem_access(cur_dmem_access),

    .rd_in       (dmem_rdata),

    .wd_in       (cur_rf_rd1),

    .rd_out      (cur_rd_out),

    .wd_out      (dmem_wdata)

);

MUX2 #(

    .WIDTH  (32)

) my_mux2(

    .src0      (cur_pc + 4),

    .src1      (cur_alu_res),

    .src2      (cur_rd_out),

    .src3      (32'h0),

    .sel       (cur_rd_wd_sel),

    .res       (cur_rf_wd)

);

assign dmem_addr = (cur_dmem_access == 4'b1000)? 0 :cur_alu_res;//判断是否存入寄存器中
```

```
// Commit

reg  [ 0 : 0]   commit_reg          ;

reg  [31 : 0]   commit_pc_reg       ;

reg  [31 : 0]   commit_inst_reg     ;

reg  [ 0 : 0]   commit_halt_reg     ;

reg  [ 0 : 0]   commit_reg_we_reg   ;

reg  [ 4 : 0]   commit_reg_wa_reg   ;

reg  [31 : 0]   commit_reg_wd_reg   ;

reg  [ 0 : 0]   commit_dmem_we_reg  ;

reg  [31 : 0]   commit_dmem_wa_reg  ;

reg  [31 : 0]   commit_dmem_wd_reg  ;

// Commit

always @(posedge clk) begin

    if (rst) begin

        commit_reg          <= 1'B0;

        commit_pc_reg       <= 32'H0;

        commit_inst_reg     <= 32'H0;

        commit_halt_reg     <= 1'B0;

        commit_reg_we_reg   <= 1'B0;

        commit_reg_wa_reg   <= 5'H0;

        commit_reg_wd_reg   <= 32'H0;

        commit_dmem_we_reg  <= 1'B0;

        commit_dmem_wa_reg  <= 32'H0;

        commit_dmem_wd_reg  <= 32'H0;

    end

    else if (global_en) begin

        commit_reg          <= 1'B1;

        commit_pc_reg       <= cur_pc;   // TODO

        commit_inst_reg     <= cur_inst;   // TODO

        commit_halt_reg     <= (cur_inst ==  32'H80000000);   // TODO
```

```
            commit_reg_we_reg   <= cur_rf_we;   // TODO
            commit_reg_wa_reg   <= cur_rf_wa;   // TODO
            commit_reg_wd_reg   <= cur_rf_wd;   // TODO
            commit_dmem_we_reg  <= dmem_we;   // TODO
            commit_dmem_wa_reg  <= dmem_addr;   // TODO
            commit_dmem_wd_reg  <= dmem_wdata;   // TODO
        end
    end
    assign commit           = commit_reg;
    assign commit_pc        = commit_pc_reg;
    assign commit_inst      = commit_inst_reg;
    assign commit_halt      = commit_halt_reg;
    assign commit_reg_we    = commit_reg_we_reg;
    assign commit_reg_wa    = commit_reg_wa_reg;
    assign commit_reg_wd    = commit_reg_wd_reg;
    assign commit_dmem_we   = commit_dmem_we_reg;
    assign commit_dmem_wa   = commit_dmem_wa_reg;
    assign commit_dmem_wd   = commit_dmem_wd_reg;
endmodule
```

本题要求根据数据通路将各个模块正确连接成一个简单的 CPU，此处用到的模块有：PC、DECODER、REG_FILE、MUX 和 ALU，其中 MUX 和 PC 均使用在实验文档中直接给出的代码，DECODER、REG_FILE 和 ALU 则是在先前实验和本次实验的前一题中实现。各个模块之间的信号传递按照数据通路进行连接即可。其中注意 ALU 的两个操作数均需要选择器前置来确定输入信号是寄存器/PC/立即数，且选择器控制信号须在 DECODER 中与 MUX 的信号先后顺序相对应。

任务 3：思考与总结（3 分）

请在实验报告中回答下面的问题：

1. 本次实验的 CPU 中，哪些模块用到了时钟信号？
2. 请分别给出一条指令，以符合下面的描述：
   - alu_src0 选择 pc；
   - alu_src0 选择 rf_rd0；
   - alu_src1 选择 rf_rd1；
   - alu_src1 选择 imm；
3. 请指出本次实验的 CPU 中可能的关键路径；如果这条路径的延迟大于一个时钟周期，可能会带来什么影响？

1. 使用时钟信号的模块：REG_FILE。

2. alu_src0 选择 pc：pcaddu12i 指令，功能为加载加上 PC 的高 20 位立即数。

   alu_src0 选择 rf_rd0：add.w 指令，功能为取两个寄存器中的数据进行加法。

   alu_src0 选择 rf_rd1：add.w 指令，功能为取两个寄存器中的数据进行加法。

   alu_src0 选择 imm：lu12i.w 指令，功能为加载高 20 位立即数。

3. 可能的关键路径：PC 输出到 DECODER 到 ALU 的路径，若其延迟大于一个时钟周期，可能会导致指令的识别和执行出现错误。

仿真结果与分析

1. 仿真结果：

```
ubuntu@VM9505-aymx9:~/Desktop/COD25-Simulation-Framework$ ./build/sim
Simulation started...
Difftest level: FULL.
Dumping waveform to waveform/waveform.vcd.
Hit good trap.
Simulation finished.
Time:                          1068
Instruction count:             533
PC:                            0x1C000850
Instruction:                   0x80000000
Registers at the end:
[ 0] = 0x00000000 [ 1] = 0x00000000 [ 2] = 0xB8231844 [ 3] = 0x5A973000
[ 4] = 0x00000DC1 [ 5] = 0x0000077D [ 6] = 0x9D692E90 [ 7] = 0x30568814
[ 8] = 0x0000077D [ 9] = 0x00000000 [10] = 0x95C44000 [11] = 0x8BD9C838
[12] = 0x73F62808 [13] = 0x8BD9CEB8 [14] = 0x00000840 [15] = 0xFFFFF6BC
[16] = 0x00000000 [17] = 0x4B61F7D8 [18] = 0x9D692820 [19] = 0x7274AEA4
[20] = 0x00000001 [21] = 0x30568E7C [22] = 0xB8231ECC [23] = 0x5A973000
[24] = 0x53BB6E2C [25] = 0xC3AD6E54 [26] = 0xFCDDF000 [27] = 0x7274A82C
[28] = 0x3FDB6E40 [29] = 0x0000086F [30] = 0x000003FE [31] = 0x00000001
ubuntu@VM9505-aymx9:~/Desktop/COD25-Simulation-Framework$
```

上板：

```
User: RR 0 10;
00000000 008D86A0 0000041F 4782B908
00000000 DF8DF928 BD966000 00000000
0000010D FB024688

00000000 008D86A0 0000041F 4782B908
00000000 DF8DF928 BD966000 00000000
0000010D FB024688 C6D4E000 00000001
00000335 A77F9000 557F567C 00000A7F
```

电路设计与分析

1. RTL 电路描述：略

测试结果与分析

1. 本次实验结果测试全部使用在线 vscode 的仿真框架和上板结果与 lars 结果对比进行检查（见仿真结果与分析），经过测试，此次实验搭建的简单 CPU 功能正常，能够执行简单的 LA32R 指令。

总结

1. 总结：本次实验搭建了一个简单的单周期 CPU,实现了各种基本的数值和逻辑运算以及两个简单的立即数操作。通过此次实验我更好地熟悉了阅读数据通路并且正确连接模块信号的能力，并且对 CPU 内部结构如 DECODER 模块等的功能有了更清晰的认识理解。